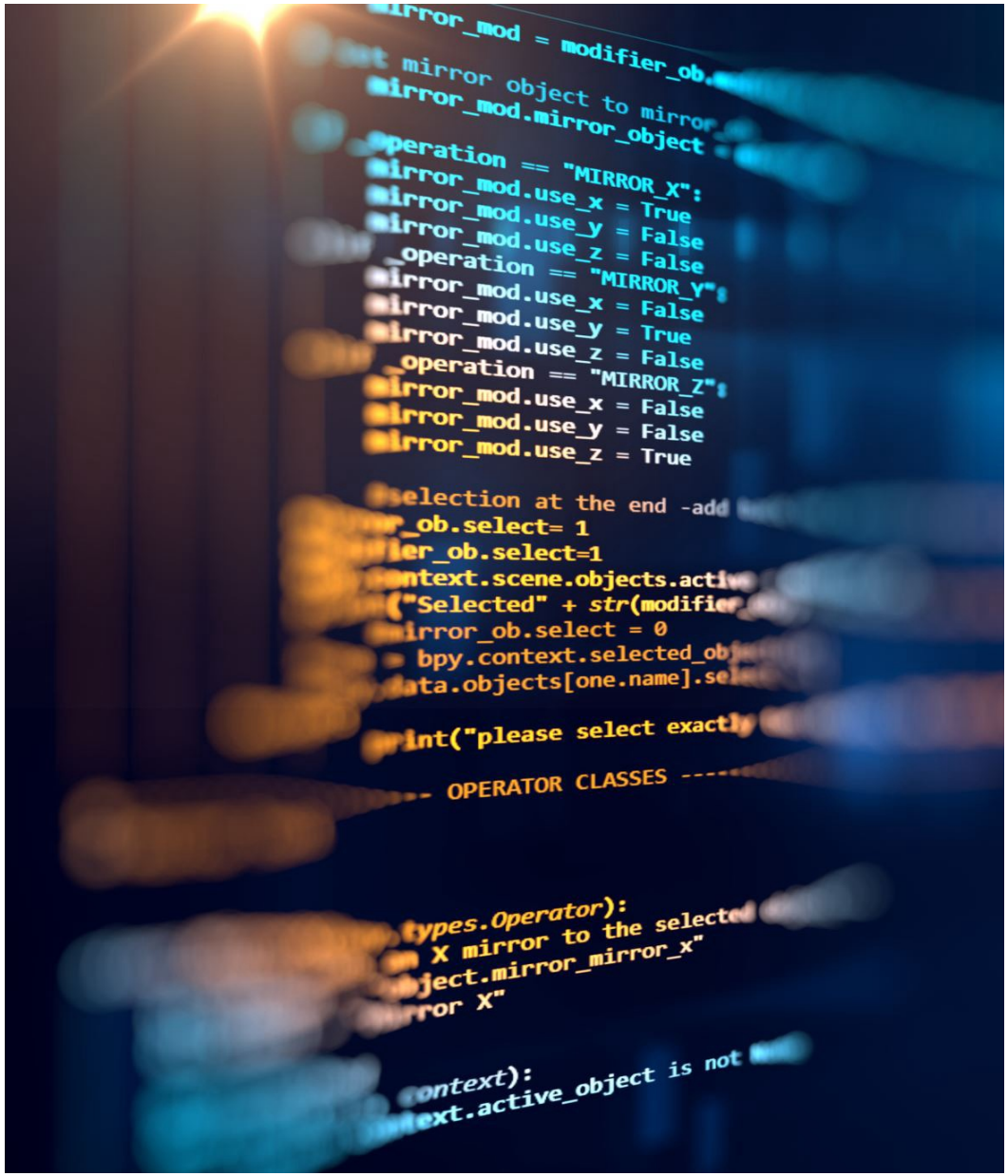
CODE SMELLS & CODE METRICS

Advanced Programming
CO2039



CONTENTS

- Code smells
- Code metrics

CODE CAN ALSO SMELL

Code smell serves an indication that there is deeper problem in the system

- Code smells are only hints – not necessarily a problem!
- Look for bad smells – definitely needs a refactoring!
- But is there some list that could be used? Or common ones?

BLOATERS

Long method: A method contains too many lines of code. Generally, any method longer than ten lines should make you start asking questions.

Large class: A class contains many fields/methods/lines of code.

Long parameter list: More than three or four parameters for a method.

Primitive obsession

Data clumps

BLOATERS: PRIMITIVE OBSESSION

- **Primitive obsession** is when you use basic data types (int, double, string, Boolean) to represent things that should be modeled as their own object or type.
- **Example**:

```
public class Order {
      private String customerEmail; private double amount;
      public Order(String customerEmail, double amount) {
            this.customerEmail = customerEmail;
            this.amount = amount;
      }
      public boolean isValidEmail() { return customerEmail.contains("@"); }
}
```

BLOATERS: DATA CLUMPS

- **Data clump** happens when the same group of variables repeatedly appear together in multiple places. If several parameters are always passed together, they probably belong in their own class.
- **Example**:

```
public class StudentService {
     public void registerStudent(String firstName, String lastName,
String street, String city, String zipCode) { // logic }
     public void updateAddress(String firstName, String lastName, String
street, String city, String zipCode) { // logic }
}
```

OBJECT-ORIENTATION ABUSERS

Switch statements: You have a complex switch operator or sequence of if statements.

Temporary field: Temporary fields get their values (and thus are needed by objects) only under certain circumstances. Outside of these circumstances, they're empty.

Refused bequest: If a subclass uses only some of the methods and properties inherited from its parents, the hierarchy is off-kilter. The unneeded methods may simply go unused or be redefined and give off exceptions.

Alternative classes with different interfaces: Two classes perform identical functions but have different method names.

CHANGE PREVENTERS

Divergent change: You find yourself having to change many unrelated methods when you make changes to a class.

Shotgun surgery: Making any modifications requires that you make many small changes to many different classes.

Parallel inheritance hierarchies: Whenever you create a subclass for a class, you find yourself needing to create a subclass for another class.

DISPENSABLES

Comments: A method is filled with explanatory comments.

Duplicate code: Two code fragments look almost identical.

Lazy class: Understanding and maintaining classes always costs time and money. So if a class doesn't do enough to earn your attention, it should be deleted.

Data class: A data class refers to a class that contains only fields and crude methods for accessing them (getters and setters).

Dead code: A variable, parameter, field, method or class is no longer used (usually because it's obsolete).

Speculative generality: There's an unused class, method, field or parameter.

COUPLERS

- **Feature envy**: A method accesses the data of another object more than its own data.
- **Inappropriate intimacy**: One class uses the internal fields and methods of another class.
- **Message chains**: In code you see a series of calls resembling `$a->b()->c()->d()`
- **Middle man**: If a class performs only one action, delegating work to another class, why does it exist at all?

CODE COMPLEXITY

"The ratio of time spent reading versus writing is well over 10 to 1"

--Robert C Martin

- Code over time has tendency to accumulate complexity
- Greater or larger functionality should not have direct impact on code complexity
- Unnecessary complexity affects maintainability, time to market, understandability and testability

How to manage it? - Start measuring it!

WHAT IS MEASUREMENT?

- **Entity**: can be an Object (person) or event (journey)
- **Attribute**: Feature of property of entity (height, blood pressure, etc.)
- Two types of measurement:
  - ***Direct measurement***: measurement of attribute
  - ***Indirect measurement***: measurement of attribute involves measurement of some other attribute (e.g.: BMI)
- Uses of measurement - Assessment or Prediction

COMMONLY USED SOURCE CODE METRICS

Lines of Code (LOC)

- Easiest but effective indicator of complexity
- Small modules have low defect rates as opposed to large ones

Cyclomatic Complexity

- Developed by Thomas McCabe, 1976
- Allows to measure the complexity with respect to control flow of the code

Halstead Software Science Metrics

- Developed by Halstead, 1977
- Measures complexity in terms of the amount of information in source code

There are also object-oriented metrics (Chidamber and Kemerer 1994, Li and Henry 1993)

CYCLOMATIC COMPLEXITY

Count of the number of linearly independent paths in a program

Has a big impact on testing - test cases needs to cover the different paths

Uses the control flow graph, *G* of the given program - Approach based on graph theory

$V(G) = e - n + 2p$

e = Number of edges
n = Number of nodes
p = Connected components

In practice the number boils down to 1 (base) + number of decision points

CYCLOMATIC COMPLEXITY - EXAMPLE

```
public void displayDetails(Student student) {
       name = student.getName();
       id = student.getId();
       System.out.println(name + “ ” + id);
}
```

- Visualization: Start → getName → getId → print → Exit
- Complexity = 4 – 5 + 2 * 1 = 1

CYCLOMATIC COMPLEXITY – YOUR TURN

```
public void displayDetails(Student student) {
	name = student.getName();
	id = student.getId();
	type = student.getType();
	if (type.equals(“PGSSP”)) {
		System.out.println(name + “ ” + id + “ ” + “PGSSP”);
	} else {
		System.out.println(name + “ ” + id);
	}
}
```

Complexity = ?

HALSTEAD SOFTWARE SCIENCE METRICS

- Considers program as a collection of tokens
- Tokens: Operators or operands
- The metrics makes use of the occurrence of operators and operands in a program to reason about complexity
  - n_1 → number of distinct operators (+, -, *, while, for, (), {}, function calls, etc.)
  - n_2 → number of distinct operands (variables, method names, etc.)
  - N_1 → total number of occurrence of operators
  - N_2 → total number of occurrence of operands
- The above observations are combined to provide different metrics
  - Vocabulary $n = n_1 + n_2$
  - Program length $N = N_1 + N_2$
  - Volume $V = N * log_2(n)$

HALSTEAD SOFTWARE SCIENCE METRICS – YOUR TURN

```
public double calculateTotalCost(int item1, int item2) {
	int sum;
	final double tax = 0.12;
	sum = item1 + item2;
	double totalCost = sum * tax;
	return totalCost;
}
```

Volume = ?

THANK YOU FOR LISTENING